UNIBALSAS · CENTRO UNIVERSITÁRIO

Workshop · Desenvolvimento de APIs REST e Integração de Sistemas com Laravel

MATERIAL PRÉVIO

Preparação do Ambiente

```
PS> php -v
PS> composer -V
PS> laravel --version
PS> cloudflared --version
```

o que instalar, como conferir cada ferramenta e como criar a conta de testes no Asaas, para que o tempo do workshop seja inteiro dedicado a programar.

Para fazer antes do primeiro encontro

WORKSHOP
APIs REST e Integração com Laravel

PROFESSOR
Alex Rogaleski Marques

CONTATO
alex.rogaleski@prof.unibalsas.edu.br

1

ANTES DE COMEÇAR

Por que preparar o ambiente antes

No workshop vamos construir, do zero, uma **API REST para inscrições em eventos** com o framework **Laravel** e, no segundo encontro, ligá-la a um serviço real de pagamentos, o **Asaas**, em ambiente de testes. São dois encontros de prática contínua. Uma instalação que trava por causa de um download lento ou de uma permissão do Windows pode consumir metade de um deles. Este guia existe para que isso aconteça **antes**, com calma, e não durante a aula.

QUEM PRECISA FAZER O QUÊ

- ✓ **Vai usar o computador do laboratório?** As ferramentas já estarão instaladas. Faça mesmo assim a **Etapa 5** (conta sandbox do Asaas): ela é pessoal e ninguém pode fazê-la por você.
- ✓ **Vai usar o seu notebook?** Faça **todas as etapas**, na ordem, e termine com a lista de conferência da Seção 8.
- ✓ Reserve cerca de **uma hora**, com internet boa. A maior parte do tempo é download.

O que você vai instalar

QUADRO 1.1 — FERRAMENTAS DO WORKSHOP

Ferramenta	Para que serve no workshop	Obrigatória?
Laravel Herd	Instala de uma vez o **PHP**, o **Composer** (gerenciador de pacotes do PHP) e o **instalador do Laravel**.	Sim
SQLite	O banco de dados da API. **Já vem com o PHP**: não há servidor de banco para instalar.	Já incluso
VS Code + extensões	Editor de código, com as extensões *Laravel*, *PHP Intelephense* e *SQLite Viewer*.	Sim
Postman	Enviar requisições HTTP à API e ver as respostas, sem precisar de um front-end.	Sim
cloudflared	Criar um endereço público temporário para o Asaas conseguir avisar a sua API (webhook), no segundo encontro.	Sim
Conta sandbox Asaas	Ambiente de testes de uma plataforma de pagamentos real: cobranças, Pix e boletos de mentira.	Sim

O caminho principal deste guia é o **Windows 10 ou 11**, que é o sistema dos laboratórios. Quem usa macOS ou Linux encontra, em cada etapa, um box com as diferenças.

Roteiro da preparação

1	**PHP, Composer e Laravel.** Instalação do Laravel Herd e conferência das três ferramentas.
2	**Editor.** VS Code e as três extensões do workshop.
3	**Cliente HTTP.** Postman e a primeira requisição a uma API pública.
4	**Túnel.** Instalação do cloudflared.
5	**Asaas sandbox.** Criação da conta de testes e da chave de API.
6	**Teste final.** Um projeto Laravel de verdade rodando na sua máquina.
7	**Conferência.** Lista de verificação e solução dos problemas mais comuns.

Versões usadas na preparação deste material: **PHP 8.5**, **Composer 2.11**, **Laravel 13** (instalador 5.32), **cloudflared 2026.10**. Versões mais novas funcionam; o Laravel 13 exige **PHP 8.3 ou superior**.

2

ETAPA 1 · PHP, COMPOSER E LARAVEL

Laravel Herd

DEFINIÇÃO — O QUE É O HERD

O **Laravel Herd** é um ambiente de desenvolvimento PHP gratuito para Windows e macOS, mantido pela equipe do Laravel. Em um único instalador, ele traz o PHP, o Composer, o instalador do Laravel, o Node.js e um servidor web (Nginx). Sem ele, cada uma dessas peças seria instalada e configurada à parte.

PASSO 1.1 — BAIXAR E INSTALAR

1. Acesse `https://herd.laravel.com/windows` e baixe o instalador.
2. Execute o instalador **como administrador** (clique com o botão direito → **Executar como administrador**). O Herd precisa dessa permissão para registrar um serviço que edita o arquivo `hosts` do Windows.
3. Ao abrir o Herd pela primeira vez, siga a configuração inicial (*onboarding*): ele baixa a versão mais recente do PHP e do Node.js. Aguarde terminar.
4. A versão **gratuita** basta. A versão Pro acrescenta bancos MySQL e PostgreSQL, que não usaremos.

Durante a instalação o Herd cria a pasta `%USERPROFILE%\Herd` (por exemplo, `C:\Users\maria\Herd`). É nela que criaremos o projeto do workshop.

PASSO 1.2 — CONFERIR NO TERMINAL

Feche qualquer terminal que estiver aberto e abra um **PowerShell novo** (os programas recém-instalados só aparecem em terminais abertos depois da instalação). Execute os três comandos:

```
PS> php -v
PS> composer -V
PS> laravel --version
```

TERMINAL

```
PHP 8.5.12 (cli) (built: Oct 21 2026 22:50:07) (NTS)
Composer version 2.11.0 2026-10-29 10:23:20
Laravel Installer 5.32.0
```

Os números de versão na sua máquina podem ser diferentes. O que importa: o PHP precisa ser **8.3 ou superior** e nenhum dos três comandos pode responder *"não é reconhecido como nome de cmdlet"*.

E O BANCO DE DADOS?

O workshop usa o **SQLite**, um banco de dados que fica inteiro em **um único arquivo** dentro do projeto (`database/database.sqlite`). Não há servidor para instalar, iniciar ou configurar com usuário e senha: o PHP do Herd já traz o driver. É o banco padrão de todo projeto Laravel novo, e a escolha deixa o tempo do workshop para o que interessa, que é a API. Trocar por MySQL ou PostgreSQL mais tarde é questão de alterar quatro linhas do arquivo `.env`, porque o código da aplicação não muda.

MACOS E LINUX

macOS: baixe o Herd em `https://herd.laravel.com`; o restante é igual (a pasta é `~/Herd`). **Linux:** o Herd não existe para Linux. Use o script oficial indicado na documentação do Laravel, que instala PHP, Composer e o instalador do Laravel, e depois reabra o terminal:

```
$ /bin/bash -c "$(curl -fsSL https://php.new/install/linux/8.5)"
```

CONFIRA ANTES DE SEGUIR

Os três comandos do Passo 1.2 responderam com um número de versão. Se o PowerShell disser que `php` não é reconhecido, feche **todas** as janelas de terminal (inclusive o terminal do VS Code, se já estiver aberto) e tente de novo; se persistir, reinicie o computador.

3

ETAPA 2 · EDITOR

VS Code e extensões

Qualquer editor serve para escrever PHP, mas o **Visual Studio Code**, com as extensões certas, completa nomes de classes e métodos, aponta erros enquanto você digita e deixa navegar do nome de uma rota até o controller que ela chama. Em um workshop curto, isso poupa muitos erros de digitação.

PASSO 2.1 — INSTALAR O VS CODE

Baixe em `https://code.visualstudio.com` e instale com as opções padrão. Na tela de opções adicionais, marque **Adicionar ao PATH** e **Adicionar a ação "Abrir com Code"**: a primeira permite abrir um projeto com o comando `code .`.

PASSO 2.2 — INSTALAR AS EXTENSÕES

Abra o painel de extensões (Ctrl + Shift + X), pesquise cada nome do Quadro 3.1 e clique em **Install**. Confira o **autor:** existem extensões com nomes parecidos.

QUADRO 3.1 — EXTENSÕES DO WORKSHOP

Identificador	Nome / autor	O que faz
`laravel.vscode-laravel`	Laravel / Laravel	Extensão oficial: autocompletar rotas, views, configurações e variáveis do `.env`; ir da rota ao controller.
`bmewburn.vscode-intelephense-client`	PHP Intelephense / Intelephense	Entende a linguagem PHP: autocompletar, verificação de erros, "ir para definição". Os recursos usados aqui são gratuitos.
`qwtel.sqlite-viewer`	SQLite Viewer / Florian Klampfer	Abre o arquivo `database.sqlite` como uma planilha, para ver as tabelas e os registros.

Quem preferir o terminal instala as três de uma vez:

```
PS> code --install-extension laravel.vscode-laravel
PS> code --install-extension bmewburn.vscode-intelephense-client
```

```
PS> code --install-extension qwtel.sqlite-viewer
```

PASSO 2.3 — DESLIGAR O SUPORTE NATIVO A PHP DO VS CODE

A documentação do Intelephense recomenda desativar o recurso de PHP que já vem embutido no VS Code, para que os dois não exibam sugestões repetidas. No painel de extensões, pesquise `@builtin php`, localize **PHP Language Features** e clique em **Disable**. Mantenha ativa a **PHP Language Basics**, responsável pelas cores da sintaxe.

CONFIRA ANTES DE SEGUIR

No painel de extensões, a busca `@installed` mostra *Laravel*, *PHP Intelephense* e *SQLite Viewer*, e `@builtin php` mostra *PHP Language Features* como desativada.

ETAPA 3 · CLIENTE HTTP

4 Postman e a primeira requisição

DEFINIÇÃO — CLIENTE HTTP

Uma API não tem tela: ela recebe **requisições** e devolve **respostas**, normalmente em JSON. Um **cliente HTTP** é o programa que monta essas requisições à mão (método, endereço, cabeçalhos, corpo) e mostra a resposta. O navegador faz isso o tempo todo, mas só com o método GET e sem deixar controlar os detalhes. O **Postman** permite controlar tudo.

PASSO 3.1 — INSTALAR E CRIAR A CONTA

1. Baixe o aplicativo de desktop em `https://www.postman.com/downloads` e instale.
2. **Crie uma conta gratuita e entre nela.** Sem conta, o Postman abre num modo simplificado que envia requisições, mas **não salva coleções nem ambientes**, e usaremos os dois.

PASSO 3.2 — CONSULTAR UMA API PÚBLICA

Vamos conversar com uma API de verdade: o **ViaCEP**, que devolve o endereço de um CEP. No Postman, clique em **New → HTTP**, deixe o método em `GET`, cole o endereço abaixo e clique em **Send**:

GET https://viacep.com.br/ws/65800000/json/ 200 OK

RESPOSTA

```
{
  "cep": "65800-000",
  "logradouro": "",
  "complemento": "",
  "unidade": "",
  "bairro": "",
  "localidade": "Balsas",
  "uf": "MA",
  "estado": "Maranhão",
  "regiao": "Nordeste",
  "ibge": "2101400",
  "gia": "",
  "ddd": "99",
  "siafi": "0727"
}
```

Repare em três coisas que vão aparecer o tempo todo no workshop: o **método** (`GET`, "buscar"), o **código de status** (`200 OK`, "deu certo") e o **corpo** da resposta, em **JSON**: pares de *chave* e *valor* entre chaves. Troque o CEP pelo da sua rua e envie de novo; depois tente um CEP inexistente, como `00000000`, e observe a resposta.

PARE E PENSE

1. O ViaCEP não sabe se quem pediu foi o Postman, um aplicativo de celular ou outro servidor. Por que isso é uma vantagem?
2. Na aba **Headers** da resposta, procure `Content-Type`. O que ele informa?
3. Abra o mesmo endereço no navegador. O que muda e o que continua igual?

5

ETAPA 4 · TÚNEL

cloudflared

No segundo encontro, o Asaas vai precisar **chamar a sua API** para avisar que um pagamento foi feito. Só que a API estará rodando no seu computador, num endereço como `http://127.0.0.1:8000`, que só existe dentro da sua máquina. O **cloudflared** resolve isso criando um **túnel**: um endereço público e temporário, do tipo `https://palavras-aleatorias.trycloudflare.com`, que encaminha tudo o que recebe para a sua API local. O modo que usaremos (*quick tunnel*) **não exige conta**.

PASSO 4.1 — INSTALAR E CONFERIR

No PowerShell, use o gerenciador de pacotes do Windows (`winget`). Depois, **abra um terminal novo** e confira:

```
PS> winget install --id Cloudflare.cloudflared
PS> cloudflared --version
```

TERMINAL

```
cloudflared version 2026.10.7 (built 2026-10-30-17:05 UTC)
```

Se o `winget` não estiver disponível, baixe o executável `cloudflared-windows-amd64.exe` na página de downloads do Cloudflare Tunnel, renomeie para `cloudflared.exe` e coloque-o numa pasta que esteja no PATH (ou na pasta do projeto). No macOS, `brew install cloudflared`; no Linux, o pacote `.deb` ou `.rpm` da mesma página.

NÃO É PRECISO ABRIR O TÚNEL AGORA

Basta o comando responder com a versão. O túnel será aberto no workshop, quando a API já tiver uma rota para receber os avisos do Asaas.

6

ETAPA 5 · PLATAFORMA DE PAGAMENTOS

Conta sandbox do Asaas

DEFINIÇÃO — SANDBOX

Um **sandbox** (“caixa de areia”) é uma cópia de um sistema real feita para testes: tem as mesmas telas e a mesma API da versão de produção, mas **o dinheiro não é real**. No sandbox do Asaas é possível criar clientes, emitir cobranças Pix e boleto e simular que elas foram pagas, sem que ninguém pague nada. É assim que os desenvolvedores integram um sistema de pagamentos antes de colocá-lo no ar.

PASSO 5.1 — CRIAR A CONTA

1. Acesse `https://sandbox.asaas.com/login/auth` e escolha a opção de **criar conta**. A conta do sandbox é **separada** da de produção: mesmo quem já tem conta no Asaas precisa criar outra aqui.
2. Preencha os dados cadastrais. Eles **não precisam ser reais**, mas precisam ter **formato válido** (um CPF ou CNPJ com dígitos verificadores corretos, um CEP que exista). A própria documentação do Asaas sugere usar um gerador de dados de teste, como o site *Gerador BR*.
3. No **nome**, use apenas letras e espaços, sem números nem símbolos: `Maria Teste Workshop`, e não `Maria_Teste01`. Segundo a documentação, um nome fora desse padrão pode impedir a aprovação automática da conta e deixar o Pix desabilitado.
4. Use o **seu e-mail de verdade**: é com ele que você vai entrar depois.

5. Quando o sandbox pedir um **código de confirmação** (de e-mail, SMS ou ativação), digite `000000`. No ambiente de testes esse código substitui os que seriam enviados.

Com os dados obrigatórios preenchidos corretamente, a conta sandbox é **aprovada automaticamente**.

PASSO 5.2 — GERAR A CHAVE DE API

A **chave de API** é a "senha" que a sua aplicação vai apresentar ao Asaas em cada requisição. Com a conta aberta no sandbox:

1. Abra o menu do usuário e vá em **Integrações → Chaves de API**.
2. Gere uma nova chave. Dê a ela um nome que identifique o uso, como `workshop-laravel`; a data de expiração é opcional.
3. **Copie a chave imediatamente** e guarde-a num arquivo de texto no seu computador ou num gerenciador de senhas. O Asaas mostra a chave **uma única vez**; se você a perder, terá de gerar outra.

A chave do sandbox é longa e começa sempre com `$aact_hmlg_` ("hmlg" de *homologação*). Chaves de produção começam com `$aact_prod_`: se a sua começar assim, você está na conta errada.

A CHAVE É UMA CREDENCIAL

- ✓ Quem tem a chave age em nome da sua conta. **Não** a envie em grupos, não a cole em fóruns nem em ferramentas on-line e não a coloque em repositórios do GitHub.
- ✓ No projeto, ela vai morar no arquivo `.env`, que o Laravel já deixa fora do Git.
- ✓ Mesmo no sandbox, o Asaas **pode enviar e-mails e SMS de verdade** aos clientes cadastrados. Nos testes, nunca use e-mail ou telefone de outras pessoas.

PASSO 5.3 — TESTAR A CHAVE NO POSTMAN

Antes do workshop, confirme que a chave funciona fazendo uma consulta à API do sandbox. No Postman, crie uma requisição `GET` para o endereço abaixo e, na aba **Headers**, acrescente os dois cabeçalhos do Quadro 6.1:

GET	https://api-sandbox.asaas.com/v3/customers	200 OK

QUADRO 6.1 — CABEÇALHOS EXIGIDOS PELA API DO ASAAS

Key	Value	Por quê
`access_token`	a sua chave `$aact_hmlg_…`	Autenticação. O Asaas **não** usa o padrão `Authorization: Bearer`.
`User-Agent`	`workshop-laravel`	Identifica a aplicação; obrigatório para contas criadas a partir de junho de 2024.

Como a conta é nova, a lista de clientes vem vazia, e é exatamente isso que se espera:

RESPOSTA

```
{
    "object": "list",
    "hasMore": false,
    "totalCount": 0,
    "limit": 10,
    "offset": 0,
    "data": []
}
```

Se a resposta for `401`, a chave foi digitada errada (falta o `$` inicial? sobrou um espaço?) ou pertence a outro ambiente. Teste também apagar o cabeçalho `access_token` e reenviar: a resposta de erro mostra como o Asaas descreve problemas, num formato que veremos de novo no workshop.

CONFIRA ANTES DE SEGUIR

Você consegue entrar em `https://sandbox.asaas.com/login/auth`, tem a chave `$aact_hmlg_…` guardada em lugar seguro e a requisição do Passo 5.3 devolveu `200`. **Leve a chave no dia do workshop**: sem ela, a segunda parte não funciona.

ETAPA 6 · TESTE FINAL

7 Um projeto Laravel rodando

A última verificação é a mais completa: criar um projeto Laravel de verdade e abri-lo no navegador. Se isto funcionar, todas as peças da Etapa 1 estão conversando entre si.

PASSO 6.1 — CRIAR E INICIAR UM PROJETO DE TESTE

```
PS> cd ~\Herd
PS> laravel new teste --database=sqlite --phpunit --no-boost --no-node
PS> cd teste
PS> php artisan serve
```

As opções do segundo comando respondem de antemão às perguntas do instalador (banco SQLite, PHPUnit para testes, sem Laravel Boost e sem Node). Se mesmo assim ele perguntar por um *starter kit*, escolha **None**. A criação leva de um a três minutos, porque o Composer baixa as dependências. O último comando sobe o servidor de desenvolvimento e fica rodando:

```
TERMINAL
   INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server
```

Abra `http://127.0.0.1:8000` no navegador: deve aparecer a página de boas-vindas do Laravel. Volte ao terminal e pare o servidor com Ctrl + C.

Como o projeto está na pasta do Herd, ele também fica acessível, sem `artisan serve`, em `http://teste.test`. No workshop usaremos o `artisan serve`, que funciona igual em qualquer sistema, mas os dois endereços servem.

PASSO 6.2 — ABRIR NO VS CODE E VER O BANCO

```
PS> code .
```

No VS Code, abra `database/database.sqlite`. Com o SQLite Viewer instalado, o arquivo abre como uma lista de tabelas (`users`, `cache`, `jobs` e outras), que o Laravel criou sozinho ao gerar o projeto. Depois do teste, a pasta `teste` pode ser apagada.

ETAPA 7 · CONFERÊNCIA

8 Lista de verificação e problemas comuns

Está tudo pronto se...

- [ ] `php -v` mostra PHP 8.3 ou superior.
- [ ] `composer -V` e `laravel --version` respondem com a versão.
- [ ] O VS Code tem as extensões *Laravel*, *PHP Intelephense* e *SQLite Viewer*.
- [ ] O Postman está instalado, você entrou na sua conta e a consulta ao ViaCEP funcionou.
- [ ] `cloudflared --version` responde com a versão.
- [ ] Você entra no sandbox do Asaas e tem a chave `$aact_hmlg_…` guardada.
- [ ] A consulta `GET /v3/customers` ao sandbox devolveu `200`.
- [ ] O projeto de teste abriu a página de boas-vindas em `http://127.0.0.1:8000`.

QUADRO 8.1 — PROBLEMAS COMUNS E SOLUÇÕES

Sintoma	Causa provável e solução
`php` (ou `composer`, `laravel`) "não é reconhecido"	O terminal foi aberto antes da instalação. Feche todos os terminais, inclusive o do VS Code; se persistir, reinicie o Windows.
Herd não instala ou não inicia	Faltou executar o instalador **como administrador**. Desinstale e repita.
`laravel new` muito lento ou com erro de download	Rede instável ou bloqueada. Tente outra rede. O antivírus pode atrasar: a documentação do Herd sugere excluir a pasta `%USERPROFILE%\.config\herd` da verificação do Windows Defender.
`Failed to listen on 127.0.0.1:8000`	A porta já está em uso (outro `artisan serve` aberto?). Feche-o ou use `php artisan serve --port=8001`.
Script bloqueado no PowerShell (*execution policy*)	Execute uma vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` e abra um terminal novo.
Asaas responde `401`	Chave incompleta (sem o `$` inicial, com espaço sobrando), de produção em vez de sandbox, ou cabeçalho com nome errado (é `access_token`, com sublinhado).
Pix desabilitado no sandbox	Dados cadastrais incompletos ou fora do formato. Revise o cadastro, principalmente o nome (sem números nem símbolos).

PARA O PRIMEIRO ENCONTRO

- ✓ Traga o notebook **carregado** e com tudo da lista acima funcionando, ou use o laboratório.
- ✓ Traga a **chave de API do sandbox** e a senha da conta do Asaas.
- ✓ Saiba entrar na sua conta do **Postman**.
- ✓ Travou em algum passo? Escreva para **alex.rogaleski@prof.unibalsas.edu.br** com o print do erro, *antes* do workshop.

Referências

1. ASAAS. **Sandbox**. Documentação da API Asaas. Disponível em: https://docs.asaas.com/docs/sandbox. Acesso em: nov. 2026.
2. ASAAS. **Chaves de API**. Documentação da API Asaas. Disponível em: https://docs.asaas.com/docs/chaves-de-api. Acesso em: nov. 2026.
3. CLOUDFLARE. **Cloudflare Tunnel: downloads**. Disponível em: https://developers.cloudflare.com/tunnel/downloads/. Acesso em: nov. 2026.
4. LARAVEL. **Installation**. Laravel 13.x Documentation. Disponível em: https://laravel.com/docs/13.x/installation. Acesso em: nov. 2026.

5. LARAVEL. **Herd for Windows: installation**. Disponível em: https://herd.laravel.com/docs/windows/getting-started/installation. Acesso em: nov. 2026.
6. POSTMAN. **Postman API Client**. Learning Center. Disponível em: https://learning.postman.com/docs/getting-started/basics/using-api-client. Acesso em: nov. 2026.

Material de uso didático, elaborado para o workshop *Desenvolvimento de APIs REST e Integração de Sistemas com Laravel*. Marcas e serviços citados pertencem aos respectivos titulares.